



Contents lists available at ScienceDirect

Journal of Network and Computer Applications

journal homepage: www.elsevier.com/locate/jnca

A hybrid machine learning approach for malicious behaviour detection and recognition in cloud computing

Mahdi Rabbani^{a,*}, Yong Li Wang^{a,**}, Reza Khoshkangini^b, Hamed Jelodar^a,
Ruxin Zhao^a, Peng Hu^a

^a School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, 210094, PR China

^b Center for Applied Intelligent Systems Research (CAISR), Halmstad University, Sweden

ARTICLE INFO

Keywords:

Malicious behaviour recognition
Intrusion detection system
Particle swarm optimization
Probabilistic neural network

ABSTRACT

The rapid growth of new emerging computing technologies has encouraged many organizations to outsource their data and computational requirements. Such services are expected to always provide security principles such as confidentiality, availability and integrity; therefore, a highly secure platform is one of the most important aspects of Cloud-based computing environments. A considerable improvement over traditional security strategies is achieved by understanding how malware behaves over the entire behavioural space. In this paper, we propose a new approach to improve the capability of Cloud service providers to model users' behaviours. We applied a particle swarm optimization-based probabilistic neural network (PSO-PNN) for the detection and recognition process. In the first module of the recognition process, we meaningfully converted the users' behaviours to an understandable format and then classified and recognized the malicious behaviours by using a multi-layer neural network. We took advantage of the UNSW-NB15 dataset to validate the proposed solution by characterizing different types of malicious behaviours exhibited by users. Evaluation of the experimental results shows that the proposed method is promising for use in security monitoring and recognition of malicious behaviours.

1. Introduction

A significant effort to develop new computing technologies has been undertaken in the last decade by companies and scholars. Cloud computing (Mell/Tim Granceet al., 2011), mobile computing, fog computing (Bonomi et al., 2012), edge computing and IoT (Shi et al., 2016) have created a very complicated digital world, in which users can easily execute a variety of tasks in a short time and at low cost. These computing technologies provide different services, including resource pooling, broad network access, on-demand self-service, measured service and rapid elasticity (Hoefer and Karagiannis, 2010). While such technologies have been very helpful in creating and improving new services, developing a secure Cloud-based environment is still a major concern (Alouane and El Bakkali, 2015). Cyberattacks and emerging security threats are increasing in prominence daily, penetrating networks, influencing major elections, affecting servers and crippling businesses and financial accounts overnight (Monowar et al., 2014). In fact, these

events create concern among investors that cyber threats are the greatest threat to their operations. This leads to an important question: *if a full cryptographic solution is not currently achievable, what would a near-term solution look like?* A practical solution needs to be effective enough to change users' view of the Cloud from a black box interface to a justified secure platform.

Although traditional security systems (Binu and Misbahuddin, 2013) are trustable with respect to the identification of users' credentials, a malicious behaviour recognition problem enters the picture when the system needs to recognize users' abnormal behaviours at levels other than the log-in level. More to the point, traditional security techniques are facing a race between malware creators and malware defenders; current malware defences are insufficient for unknown malware because the unknown malware is often designed using novel attacking techniques (Tian et al., 2010).

A system can be considered a *secured system* when the all three principles of integrity, availability and confidentiality are comprehensively

* Corresponding author.

** Corresponding author.

E-mail addresses: rabbani@njust.edu.cn (M. Rabbani), Yongliwang@njust.edu.cn (Y.L. Wang), reza.khoshkangini@hh.se (R. Khoshkangini), jelodar@njust.edu.cn (H. Jelodar), nnzhaoms@126.com (R. Zhao), hupeng1117@163.com (P. Hu).

<https://doi.org/10.1016/j.jnca.2019.102507>

Received 14 May 2019; Received in revised form 8 October 2019; Accepted 13 December 2019

Available online 19 December 2019

1084-8045/© 2019 Elsevier Ltd. All rights reserved.

fulfilled (Moustafa and HuJill Slay, 2019). Some of the existing technologies are not securely designed; therefore, it is essential to consider the lack of security as a critical challenge in new computing technologies.

In practice, the ultimate effect of an attack can be attributed to a set or sequence of activities that is able to compromise the main features of Cloud computing environments. Indeed, analysing user behaviours is a complicated procedure since the nature of these behaviours is highly related to the dynamic context of the behaviour; the context includes aspects such as time, the location of the source and the destination, the interfaced machine, and the operating system, etc. Thus, concatenation of these factors enables the system to characterize users' behaviours and highlight multiple evidences. The evidences can then be assessed and measured in a mathematical fashion, leading to the development of better and more accurate methods for distinguishing normal from malicious behaviours.

In this context, users' behaviours, including both normal and malicious ones, are defined and represented using the accumulated packets throughout a network. The duty of a malware analysis is not only to classify samples as normal (1) or malicious (0) but also to assess the type of abnormality they may represent. We believe that a combination of traditional security systems with machine learning techniques can provide a new intrusion protection system that is meaningfully and mathematically justified and provides a secure platform for all users.

Thus, in this study we propose a novel malicious behaviour detection and recognition system that includes two main modules, *Data Pre-processing* and *Recognition*. The former is designed for data preparation and is used to extract informative features for learning and modelling purposes. The latter module is the core of the proposed technique for behavioural recognition; it includes two phases, *Training* and *Prediction*. In the second module, we used a new optimized *probabilistic neural network* to simultaneously train and optimize the network using various types of normal and malicious patterns. In fact, an optimized learning algorithm that analyses the model obtained from abnormal observations and uses this knowledge to classify new, unseen instances of malicious behaviour has been implemented (Tian et al., 2010). The proposed approach has been evaluated on the UNSW-NB15 dataset, which was captured from thousands of users in real-time network traffic (MoustafaJill Slay, 2015b).

In general, the work presented in this study presents an effective solution that can be used to automatically identify instances of malicious behaviour. This work was motivated by the following main research question: ***to what extent can we design a framework that effectively enables the detection and recognition of users' abnormal behaviour in a Cloud computing context?***

This work makes four main contributions to the field of security in the Cloud computing domain, as follows:

- We have proposed a new solution to improve security in emerging computing technologies based on observation of the users' behaviour rather than on the users' credentials.
- We have designed a hybrid machine learning system that is capable of extracting meaningful information and users' behavioural patterns from network traffic in a self-optimized fashion and using them to detect malicious behaviours in the cyber environment.
- We have developed a recognition system to classify malicious behaviours that are automatically logged in the form of vectors of features to various types of contemporary attacks.
- A performance evaluation of our proposed technique is examined in the UNSW-NB15 dataset, a well known state-of-the-art Network Intrusion Detection dataset that includes contemporary patterns for malicious evidences in modern attacks.

To the best of our knowledge, this work is novel in that it presents a way to automatically recognize abnormal behaviours based on observations captured from network traffic samples. We evaluated the performance of our proposed recognition technique using UNSW-NB15

dataset that contains a hybrid of real modern normal activities and synthetic contemporary attack instances. The experimental results indicate that our approach is promising. The remainder of this paper is organized as follows. In Section 2, we review and discuss previous works related to this area of research. The key notions of normal and malicious behaviours in the Cloud environments are reported in Section 3. Section 4 is dedicated to the proposed recognition methodology. In Section 5, the evaluation of the method and the experimental results are reported, and concluding remarks and ideas for possible extension of this work are presented in Section 6.

2. Related works

During the last decade, researchers have proposed many techniques for attenuating security intrusions (Intrusion Detection Systems) and few techniques for anomaly detection. However, with the high speed and complexity of contemporary malicious behaviours and the size of current networks, these methodologies still face the challenge of building a scalable, adaptable, and lightweight abnormal behaviour detection scheme (Moustafa and HuJill Slay, 2019), (MoustafaJill Slay and Creech, 2017). In this section, we review the state of the art in anomaly detection and indicate the position of our solution within it. Below, we report current studies that have attempted to implement malicious behaviour recognition based on machine learning, hybrid learning and other statistical approaches.

Considering the huge numbers of samples of various types of malicious behaviour that are available, machine learning approaches play an important role in automating behaviour analysis. After extracting informative features from the network's packets, machine learning algorithms can be exploited to build recognition systems. The existing research results show that machine learning approaches can perform well in detecting malicious behaviours (Rieck et al., 2008), (Tan et al., 2015), (Tian et al., 2010).

A typical malicious behaviour recognition system consists of four modules: a data source, data pre-processing, a recognition engine, and a defence response. The data source acquires portions of the raw network traffic, while data pre-processing or data preparation involves the creation of significant features that are then passed to the recognition engine and used to discriminate between normal and malicious activities (MoustafaJill Slay and Creech, 2017).

Nour Moustafa et al. (MoustafaJill Slay and Creech, 2017), proposed a Geometric Area Analysis (GAA) method for detecting anomaly patterns by using Trapezoidal Area Estimation (TAE). This method was executed on features collected from the UNSW-NB15 dataset and Beta Mixture Model (BMM) was used to compute all network parameters and to calculate the distances between observations. Using normal observations, the authors created a normal profile, and observations with certain deviations were recognized as abnormal patterns. Moreover, they used principal component analysis (PCA) to reduce the dimension of original data in network connections.

Regarding the security and privacy in Cloud-based medical infrastructures, Yuan et al. (Zhang et al., 2018) proposed a secure encrypted Electronic Medical Records (EMRs) deduplication scheme for Cloud-assisted eHealth systems (HealthDep). They integrated the analyses results and explained how Health-Dep gives ability to the Cloud server to efficiently accomplish the EMRs deduplication, and empowers the Cloud server to decrease storage costs by approximately 65 percent, while ensuring the confidentiality of EMRs.

Yuan et al. (Zhang et al., 2019) introduced the first certificateless public verification framework against procrastinating auditors (CPVPA) using blockchain technology. By using auditors they recorded each verification result in form of transaction based on blockchain technique, which gives ability to users to check whether auditors perform the verifications at the prescribed time. An interesting study on security in Cloud computing has been done by Mouna et al. (Jouini and Arfa Rabai, 2019). In this study, they developed a quantitative security risk assess-

ment framework named Multi-dimensional Mean Failure Cost (M2FC) for Cloud computing environments. This model uses a generic framework that investigates and evaluate Cloud security issues, and then proposes suitable countermeasures to solve these problems.

Mohammed et al. (Ambusaidi et al., 2016) proposed a least squares support vector machine-based IDS (LSSVM) for intrusion detection. In the feature extraction step, they proposed the flexible mutual information feature selection (FMIFS) technique, which is a supervised filter-based feature selection algorithm that is applied to decrease the redundancy between features. FMIFS obliterates the redundancy parameter that is needed in MIFS. This technique is a least-squares edition of SVM that works with equality constraints instead of inequality constraints in the formulation implemented to deal with a set of linear equations for classification problems rather than a quadratic programming problem (Ambusaidi et al., 2016).

Tan et al. (2014) proposed a Denial of Service (DoS) attack detection system by using multivariate correlation analysis (MCA) to analyse network traffic. They tried to discover geometrical correlations between normal observations and increase the progress of MCA by using triangle-area-based method. Although they achieved high-performance results, this technique can only detect DoS attacks and is not applicable to other types of attacks.

Another study proposed an artificial immune system (AIS) for anomaly detection and prevention using self-tuning of detectors and detector power for anomaly detection (Saurabh and Verma, 2016). Computer vision techniques (CVT) were used for detection of DoS attacks; in this method, network traffic records are treated as images, and detection of DoS attacks is treated as a computer vision problem. The authors used earth movers distance (EMD) to measure dissimilarities between observed objects (Tan et al., 2015).

Chih-Fong et al. proposed triangle area nearest neighbours (TANN), a hybrid learning model for attack detection. In this technique, k-means clustering is first applied to obtain cluster centres for attack classes, new feature signatures are then calculated, and finally the k-NN classifier is used to classify similar attacks based on the new features (Tsai and Lin, 2010).

In the feature extraction phase, behavioural data are transformed into digital values using specific techniques (e.g., N-grams, BRO-IDS) (Moustafa and HuJill Slay, 2019). Feature selection is then performed to reduce the dimension of the original features and thereby acquire excellent feature sets that can be used to train the model. This is done using domain knowledge or feature selection functions such as principal component analysis (PCA) (Mithal et al., 2016), Information Gain (IG) (Moonsamy and TianLynn, 2011), or the chi-square test (Mohamed and Mazouzi, 2015).

Cen et al. (2015) proposed a learning approach based on a probabilistic discriminative model for the detection of android malware. Using statistical techniques, the frequencies of API calls are counted and then truncated to either 0 or 1 using a predefined threshold to simplify the feature representation. Finally, to decrease the size of the feature space, IG and CHI functions are used for feature selection.

Zhang et al. (2014) conducted a similar study in which they proposed an Android malware classification system via dependency graphs called DroidSIFT. They used graph-based similarity to tackle the variety of malware specially Zero-day malware and discover the behaviours of homogeneous applications while tolerating minor implementation differences. The system includes graph databases from function-called APIs and produces graph-based feature vectors that bear a non-zero similarity rate in one pattern only if the corresponding graph matches one of the graphs in the database.

Watson et al. (2016) proposed an online Cloud malicious detection approach. In this study, they classified security into three levels. The first level, system-level data, includes memory usage, peak memory usage and the number of threads and handles. The second level is network-level data; it includes packets, bytes, and flows per address for each feature. The authors used mean, variance and standard deviation

to build statistical meta-features. They assess the malicious detection using a system analysis engine (SAE) and a network analysis engine (NAE) during the onset of Denial of Service (DoS) attacks. The limitation of this work is that a signature-based intrusion detection system alone is insufficient to deal with modern malware characteristics; a comprehensive intelligent defender system is required to monitor users' behaviours in Cloud-based environments.

Recently, some work has been conducted in which the behaviour of malware in different platforms were analysed. Typical malicious behaviours include spyware-like behaviours (Kirda et al., 2006), evasive behaviours (Sun et al., 2011), environment-sensitive behaviours (also called trigger-based behaviours) (Martignoni et al., 2009), and network scan behaviours (Inoue et al., 2009). The studies presented above point out the lack of security principles in currently available systems. Now it is time to construct a security system that can detect and recognize malicious behaviours at every level of a security system and not merely at the log-in level. Below, we list the main gaps and limitations identified in related works:

- The existing security systems lack self-optimization in learning as well as self-configuration and preventive capabilities (Zhao et al., 2014), (Watson et al., 2016), (Zhang et al., 2014).
- Although a few approaches achieved high-performance results for some specific attacks such as DoS, they are not applicable to other types of attacks (Tan et al., 2014), (Tan et al., 2015).
- There is a lack of collaboration between detectors in recognizing specific patterns as anomalous.
- Recognition of contemporary and Zero-day attacks are required.

3. Background

In this section, we present the main concepts of behaviour recognition in cloud-based environments through which behavioural data types are studied from a mathematical point of view.

3.1. Evidence and observations in cloud-based environments

The current rapid increase in the number of users of Cloud-based environments and the Internet in general has made the digital world much more complicated than the real world. In this digital space, normal users and cyberattackers coexist, but they engage in different types of activities. One of the main challenges in devising a system to collect and recognize these virtual human behaviours is collection of the evidence (observations) from network traffic packets. This is not a trivial task because evidences are not in a standard format; in some cases, these evidences are described qualitatively rather than presented in numerical form. Indeed, representing these behaviours in a standard format that is understandable to a machine is a challenging task that should be more fully investigated.

In Table 1, we describe some behavioural types that users generate during execution in files, networks and registries to simplify the concept of user behaviours in Cloud-based environments (Han et al., 2019).

Moreover, a user behavioural structure in a Cloud-based environment can be divided into different levels of sub-behaviours and lightweight observations. These components are illustrated in Fig. 1. In the lowest level, short-terms observations called evidences are collected as a basis for deciding about upper level sub-behaviours; subsequently, sub-behaviours define the final behaviour of a user.

3.2. Intrusion detection system (IDS)

Intrusion detection techniques are classified into two main types: misuse detection (MD) and anomaly detection (AD) (MoustafaJill Slay and Creech, 2017). MD traces network traffic to find observations similar to attack signatures available in the dataset that is used to distinguish normal behaviours from abnormal instances. In AD, the system

Table 1
High-level behaviour examples.

Behavioural type	Evidences
Behavioural operations of the files	Number of file operations such as create, recreate, open, read, write, delete, failed, move and exist; Number of folder operations such as create, remove and enumerate
Registry operations	Number of registry operations including create, read, write, open and delete
Network behavioural operations	Number of different source/destination IPs, source/destination ports, query hosts/dead hosts/domain/DNS/DNS servers by different protocols



Fig. 1. Behaviour components.

creates a normal profile and identifies any variations from it as suspicious events. Anomaly instances happen when an observation deviates from the normal profile; from a mathematical prospective, this observation is assigned to as an outlier. Contextual anomalies occur when patterns are anomalous in a particular context. Basically, they appear as related behaviours that differ from the majority of normal activities. AD systems can identify known and Zero-day attacks using less effort to construct a profile than MD systems. Support vector machine (SVM) (Kang et al., 2012) and artificial neural network (ANN) (Saber et al., 2017) are the two famous techniques in intrusion detection systems (MoustafaJill Slay and Creech, 2017).

IDS is usually used to monitor host and network-based environments. A host-based IDS (HIDS) monitors the evidences of a host by collecting information about different types of activities that take place in a computer system. In addition, a sensor should be set up in the system to monitor hosts and log the operating system's activities. Network-based IDS (NIDS) monitors network traffic to recognize remote attacks that are transferred over a network connection (Keshk et al., 2017). IDSs are deployed and installed to various systems and applications such as, data centres, access points, Cloud and Internet of Things (IoT) (Moustafa and HuJill Slay, 2019). Cloud-based IDSs usually spreaded out on the nodes of centralized networks and essential for firms that migrate services to public Cloud paradigms (i.e., Microsoft Azure and Amazon Web Services) to protect their software and infrastructures (Moustafa and HuJill Slay, 2019). In summary, existing IDSs can not successfully detect and recognize the internal malicious behaviours specially in Cloud and mobile computing environments due to their potential complexity and remotely located modules. Hence, a modern IDS is able to face with end-to-end encryption by extracting informative information about network packets. Some advantages and disadvantages of Cloud-based IDS and network-based IDS are demonstrated in Table 2.

In the following section, we describe normal and malicious behaviours of users by taking advantage of the UNSW-NB dataset (MoustafaJill Slay, 2015b), which was collected from thousands of users in a Cloud environment. We postpone the detailed description of the dataset and the experimental setup to Section 5.

3.3. Normal behaviours

Normal behaviour is assigned to users who have legitimate access to a network and engage in normal activities during utilization of the network. We describe two different vectors of features to simplify the characteristics of normal behaviour; these are depicted in Table 3.

From the 47 features shown in the table, we selected some remarkable features that demonstrate the dissimilarity between normal and malicious observations. For example, feature f_2 is the dedicated source port number (0 for malicious access), f_9 is the destination-to-source transaction bytes (0 for malicious access), f_{10} is the source-to-

destination time-to-live value (the value 254 belongs to a malicious user, whereas the value 31 belongs to a normal user), f_{15} shows the source bits per second, and f_{16} shows the destination bits per second (0 for a malicious user and 666,666 for normal behaviour). Additional information about each feature's name and description can be found in Table 5.

3.4. Contemporary network abnormalities

Here, we explain the main types of malicious behaviours; these are categorized into 9 different types of attacks.

- Attack No. 1 (Analysis): This type of attack includes various types of port scanner attacks such as spam and html file penetrations.
- Attack No. 2 (Backdoors): A backdoor is a type of malicious attack that neutralizes normal authorized procedures to access a system.
- Attack No. 3 (Denial-of-service (DoS)): In this attack, the malware tries to make a machine or an operation system unavailable to its users by temporarily disrupting the host services linked to the Internet.
- Attack No. 4 (Exploits): This attack can be a sequence of comments or a piece of software in which the attacker finds a security problem in an operating system and uses it to gain control of the entire system.
- Attack No. 5 (Fuzzers): This attack is a black box technique that attempts to interrupt a program or network by feeding it randomly generated data.
- Attack No. 6 (Generic): This is essentially a collision attack on the private keys of ciphers. The obvious case is a cipher that takes an N bit key; the generic attack takes a cipher text and attempts to decrypt it using all 2^N keys.
- Attack No. 7 (Reconnaissance): This type of attack collects information from networks and services using negative/theft methods. In a reconnaissance attack, the attacker attempts to acquire information about the target network and use it for unauthorized discovery and mapping of available VoIP systems, services or vulnerabilities.
- Attack No. 8 (Shellcode): This type of attack, also called Bash, exploits vulnerabilities that breach the command-line shell of various operating systems. In September 2014, when Shell-shock was used for the first time, many machines and appliances were vulnerable because they were infected by a remote code execution that probably authorized attackers to have full access and control (MoustafaJill Slay and Creech, 2017).
- Attack No. 9 (Worms): The attacker duplicates his own code as a means of spreading it to other systems. Usually, the code uses a network environment to propagate itself. This process begins when there is a security failure on the target computer.

Fig. 2 presents a tree diagram of abnormal behaviours that show the attack category for some modern attacks, the sub-categories and the frequency of each evidence observation.

4. Framework architecture

In this section, we present our proposed behavioural recognition system. It consists of two main modules, *Data pre-processing* and *Recog-*

Table 2
Comparison between network- and cloud-based IDS.

Environments	Advantage	Disadvantage
Cloud-based IDS	it can identify inappropriate use of internal tools of an organisation (MoonSung and Kim, 2016)	not able to be compatible to monitor various platforms, such as, DLLs for Windows OSs and API calls for Linux (Kholidy and Baiardi, 2012)
	can be used when a network's infrastructure was obfuscated by some bypassing techniques (i.e., fragmentation) (Kholidy and Baiardi, 2012) capable of verifying if an attack was successful or not	it can be exposable whenever its host server is compromised by an attacker (MoonSung and Kim, 2016)
Network-based IDS	without considering the destination's kind of operating system it can monitor network traffic over a specified network segment ((MoonSung and Kim, 2016), (Peng et al., 2016)) is transferable as it only monitors network packets over a certain network segment (Peng et al., 2016)	is not an appropriated solution while facing fast-spreading zero-day worms can not smoothly manage scalable systems with fast connected networks (Monowar et al., 2014)
	can receive information from both packet headers and packet payload (Kholidy and Baiardi, 2012)	not able to process encrypted data because it captures required information from packet headers (Moustafa and HuJill Slay, 2019) only gives an alert of the attack and not capable of verifying whether the attack was successful or not (Kumar et al., 2013)

Table 3
A brief comparative example of two vector of features belong to a normal and malicious users.

Feature vector	[...]	F_2	[...]	F_9	F_{10}	[...]	F_{15}	[...]	F_{16}	[...]	F_{44}	F_{45}	F_{46}	F_{47}
A normal behaviour	[...]	65435	[...]	178	31	[...]	546816	[...]	666666	[...]	1	1	1	1
A malicious behaviour	[...]	0	[...]	0	254	[...]	41600000	[...]	0	[...]	4	2	2	4

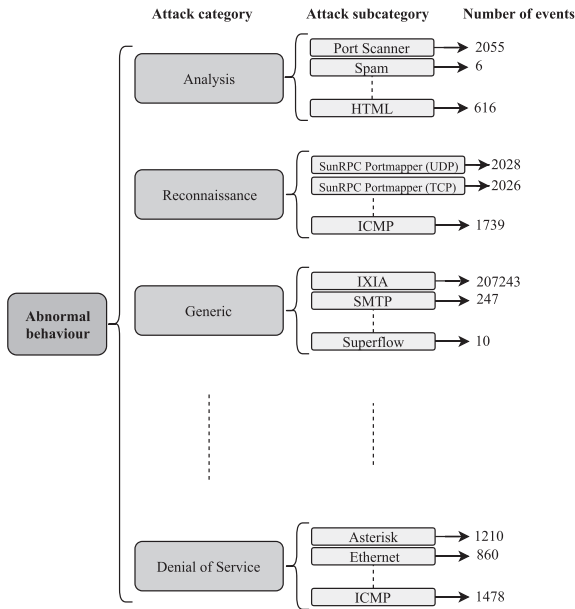


Fig. 2. The tree diagram of abnormal behaviours, sub behaviours and corresponding evidences.

tion. The first module is designed to perform data preparation and extract informative features for this purpose. The second module is the core of our proposed technique for malicious behaviour detection and recognition; its two phases, *Training* and *Prediction*, are described in detail below.

4.1. Data pre-processing module

Data pre-processing is an important step in every machine learning methods. The collected data form network packages that contain massive amounts of unnecessary and duplicated values. Pre-processing filters network data by removing irrelevant or noisy information that affects the performance of the decision engine for recognizing malicious

behaviours. Thereafter, the extracted features are compatible inputs for the recognition system. The pre-processing involves four steps: feature creation, reduction, conversion and feature normalization.

4.1.1. Feature creation

Typically, network traffic features are captured from raw network packets using tools such as Argus, BRO-IDS and Netmate (Moustafa and HuJill Slay, 2019). All collected users' observations in the UNSW-NB15 datasets have been categorized into five different groups to determine the potential characteristics of the users' behaviours, these groups include; flow features, basic features, content features, time features, generated features and additional features. These features are established using both transactional flow identifiers (i.e., source and destination IP addresses) and transactional connection times (e.g., 10 or 100 connections per second) (MoustafaJill Slay, 2015b).

4.1.2. Feature reduction

In this step, unimportant and noisy features should be removed. Feature reduction plays an important role in allowing the recognition system to effectively and efficiently detect malicious behaviours. Because network packets contain important information that might be used to identify malicious behaviours, they should be carefully analysed to separate only the relevant information that helps the recognition phase correctly detect malicious instances (Monowar et al., 2014). PCA (Abdi and Williams, 2010) is a well-known linear feature reduction technique and has the advantages of requiring less memory storage, data transfer and processing time and increasing detection accuracy compared with other methods (Abdi and Williams, 2010), (Reza Zargar and Kabiri, 2009), (Sait et al., 2015). Taking into account these advantages, we use PCA to arrange a set of variables (features) based on the highest variance for each feature and create a new low-dimensional space of uncorrelated features by removing low-variance variables (Abdi and Williams, 2010), (Reza Zargar and Kabiri, 2009).

4.1.3. Feature conversion

Features in the dataset are represented in both quantitative (i.e., numeric) and qualitative (i.e., symbolic) types. Since recognition module can process only quantitative data, a unified format is used to

convert all non-quantitative features into numeric ones. In Fig. 4, we show an example of how three symbolic features are mapped into their numeric equivalents in the UNSW-NB15 dataset. Table 4 shows the various types of data in the UNSW-NB15 dataset and the feature numbers for each category.

4.1.4. Feature normalization

This step is used to scale the feature values into a specific confidence interval. The main benefit of this step is that it removes the bias from the raw data without amending the statistical characteristics of the features. To this end, a Min-Max method (a linear transformation function (Moustafa and HuJill Slay, 2019)) has been applied for normalization, as given in equation (1).

$$X_{normalised} = \frac{(X - \min(X))}{(\max(X) - \min(X))} \quad (1)$$

In equation (1), X denotes the feature value, min (X) points the minimum value, and max (X) denotes the maximum value among all feature values in a pattern.

4.2. Recognition module

This module is the essential part of our proposed solution. It conducts a training procedure that models the users' activities and uses

the obtained information to predict whether a particular user is normal or malicious. The action flow is detailed in Figure 3. In this flow, the model that was trained using PNN is fed to the prediction component to predict the users' behaviour. The module and its corresponding components are described in detail in the following sections.

4.2.1. Training phase

In machine learning systems, networks learn from their own classification during the training process without demanding external support. Networks of this type are constructed based on competitive learning. In such a network, only one neuron among all the output neurons will pass 1 (this neuron is called the activated neuron or winning neuron), and the remaining neurons (which are called deactivated neurons) will set to 0. The proposed approach uses Particle Swarm Optimization-based Probabilistic Neural Networks (PSO-PNN) to build a self-optimized network. This specification renders PNNs faster in the training phase compared to feed-forward neural networks (Specht, 1990).

Statistically, the performance of a PNN is largely influenced by the spread parameters. We incorporated Particle Swarm Optimization (PSO) to downgrade misclassification errors and initialize with a population of random solutions and searches. Using this combined system, we then improved the standard design of PNN to a self-adaptive version. In machine learning systems, which are constructed based on PNN, the network trains using patterns with pre-defined labeled classes in

Table 4
Features with different data types in the UNSW-NB15 dataset.

Feature Type	Feature Number	Total
Binary	36,39	2
Timestamp	29,30	2
Nominal	1,3,5,6,14	5
Float	7,15,16,27,28,31,32,33,34,35	10
Integer	2,4,8,9,10,11,12,13,17,18,19,20,21,22,23,24,25,26,37,38,40,41,42,43,44,45,46,47	28
		47

Table 5
Features categories and exemplary sub-categories in the UNSW-NB15 dataset.

Category	Example of Features
Flow Features	IP address Source port number Destinations IP address Protocol type, such as TCP, UDP etc.
Basic Features	The states and its dependent protocol e.g., CON Row total duration Source to destination bytes Destination to source bytes Source to destination time to live etc.
Content Features	Source TCP window advertisement value Destination TCP window advertisement value Source TCP base sequence number. Destination TCP base sequence number mean size of the packet transmitted by the srcip etc.
Time Features	Source jitter Destination jitter Row start time Row last time Source inter-packet arrival time etc.
Generated Features	No. of methods such as Get and Post in http service If the ftp session is accessed by user and password then 1 else 0. No. of flows that has a command in ftp session No. of rows of the same service (14) and srcip (1) in 100 rows. No. of rows of the same dstip (3) in 100 rows. etc.

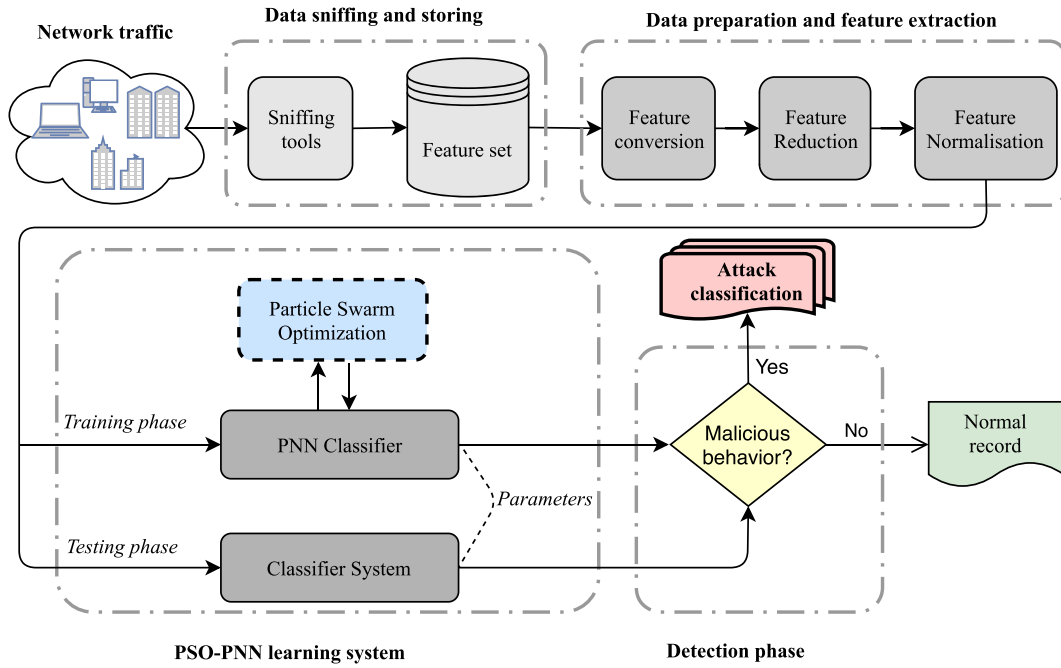


Fig. 3. Conceptual view of the proposed system.

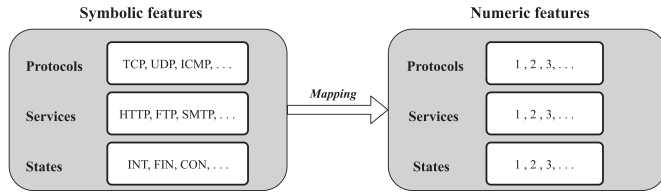


Fig. 4. Example of feature conversion using the UNSW-NB15 dataset.

dataset. Taking the extracted features into consideration at building the model, PNN can statistically find highly intra-class similarity for the observations inside every class. These statistical properties make the network capable to predict the new patterns with similar observations, that means, the system itself must learn in training phase and achieve the classification capability to classify the observations in testing phase. We trained the network to receive the final extracted features that were prepared in the pre-processing phase for all normal and malicious patterns. Finally, in the output layer of PNN, all normal patterns will be classified in one class called “normal user”, and malicious patterns will be classified as one of 9 different types of attacks (Fuzzers, Analysis, Backdoor, DoS, Exploit, Generic, Reconnaissance, Shellcode and Worm), as illustrated in Fig. 5.

The main layers of PNN are briefly described in the following.

- **Input layer**: The first layer of PNN structure is identical to the number of features in all classes that are used to model the user observations. To train the PNN system, it is necessary to feed all the nodes in the input layer with the equal numerical values from the feature vectors. As illustrated in Fig. 5, there are connections between every node in the input layer and each node in the hidden layer.

$$X = \{x_1, \dots, x_n\} \quad (2)$$

Here in Eqn 2, the feature vector for every pattern nourishes the corresponding neurons in the first layer and is then passed to the next layer.

- **Pattern layer**: All neurons in the pattern layer are distributed into k groups (one for each class). The i th neuron in the k th group calculates its output by Gaussian kernel according to the following

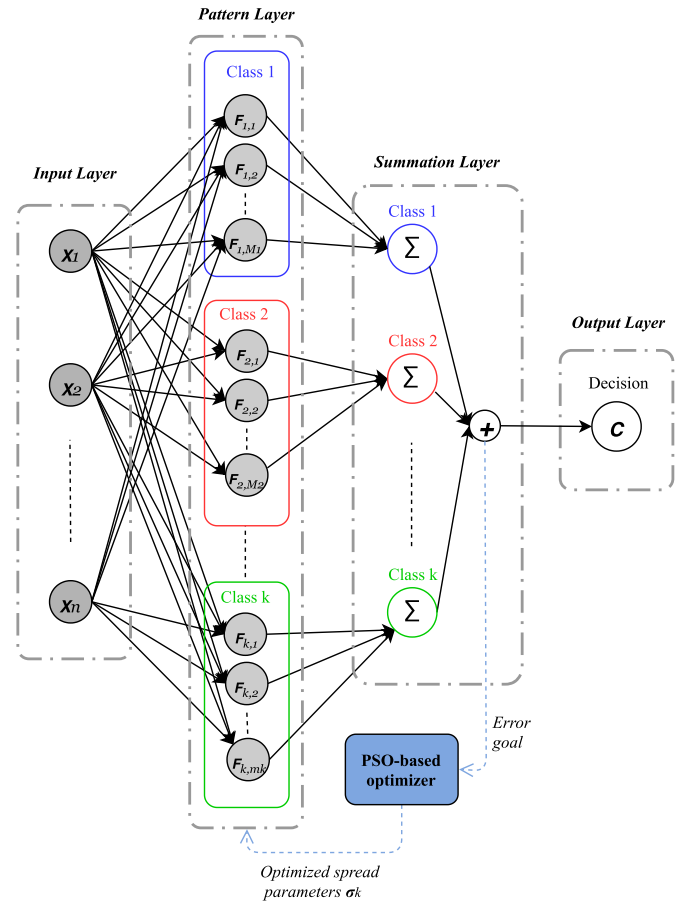


Fig. 5. A schematic illustration of PSO-PNN system.

formula:

$$F_{k,i}(X) = \frac{1}{(2\pi\sigma)^{\frac{n}{2}}} \exp\left(-\frac{\|X - X_{k,i}\|^2}{2\sigma^2}\right) \quad (3)$$

where $X_{k,i}$ is the centre of the kernel and refers to the spread parameter that determines the size of the amendable field of the kernel.

- **Summation layer:** This layer calculates the conditional functions for class probability approximation using a combination of the previously obtained densities; this function is computed using the following formula:

$$G_k(X) = \sum_{i=1}^{M_k} w_{ki} F_{ki}(X), k \in (1, \dots, K) \quad (4)$$

where M_k is the number of patterns in class k , and w_{ki} are positive coefficients satisfying $\sum_{i=1}^{M_k} w_{ki}(X) = 1$.

- **Output layer:** The last stage of the training process is the output layer; in this layer, all neurons have been processed to predict the result.

$$C(X) = \arg \max_{1 \leq k \leq K} (G_k) \quad (5)$$

Here, pattern vector X is grouped to the class that corresponds to the maximum output calculated from the summation unit.

In Fig. 5, the overall structure of a Probabilistic Neural Network (PNN), including the different layers and the complexity of each layer and its number of nodes, is schematically illustrated.

4.2.2. Particle swarm optimization phase

Particle Swarm Optimization (PSO) is a swarm-intelligence-based, nondeterministic optimization technique introduced by Dr. Eberhart (Russell and Kennedy, 1995). This algorithm was initially built to simulate the social behaviour patterns of birds flocking and fish schooling (Russell and Kennedy, 1995). As mentioned in the previous section, the spread parameter plays an important role in increasing the classification rate in PNN systems. In this hybrid technique, the determination of σ is obtained by PSO. It is initialized with a swarm value of σ , and the optimization value is computed for a specific pattern taken from the training dataset. Subsequently, the same procedure can be applied for each pattern in the training dataset to finalize a global σ ; therefore, this optimization model makes the structure of PNN a self-adaptive network. The structure of the PSO technique is twofold: *Particles* and *Swarm*. The former refers to each individual pattern, and the latter indicates the population (Clerc and Kennedy, 2002).

Each particle changes and updates its position for every movement until it finds the best position (Eberhart et al., 2001). In order to facilitate the progress of how optimization operates in our classification step in PNN, the structure of PSO and its elements, such as the local and global variants in the original search space, are simplified in the following example.

Let us consider a d -dimensional search space $S \subset S_{train}$ and a swarm including p particles; every particle i th is in the form of the vector $x_i = (x_{i1}, x_{i2}, \dots, x_{id}) \in S$. The particle's velocity for this element is $v_i = (v_{i1}, v_{i2}, \dots, v_{id}) \in S$. The best previous position achieved by the i th particle in S is denoted by $\hat{x}_i = (\hat{x}_{i1}, \hat{x}_{i2}, \dots, \hat{x}_{id})$, and g_i is the index for the particle that achieved the best past position among all of the particles in its neighbourhood area of i th particles; t is the iteration counter. A visualization graph presented in Fig. 6 (b) to figure out this process and corresponding parameters. The position of each particle can be updated by the following equation:

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (6)$$

The velocity of every particle is upgraded using the following equation:

$$v_i(t+1) = \omega v_i(t) + c_1 r_1 (\hat{x}_i(t) - x_i(t)) + c_2 r_2 (\hat{x}_{g_i}(t) - x_i(t)) \quad (7)$$

In the above equations, i is the particle index, ω is the inertial (or constriction) coefficient, c_1 and c_2 are the acceleration coefficients ($0 \leq c_1, c_2 \leq 2$), and r_1 and r_2 are the random variables ($0 \leq r_1, r_2 \leq 1$) is regenerated in every velocity update. Alternatively,

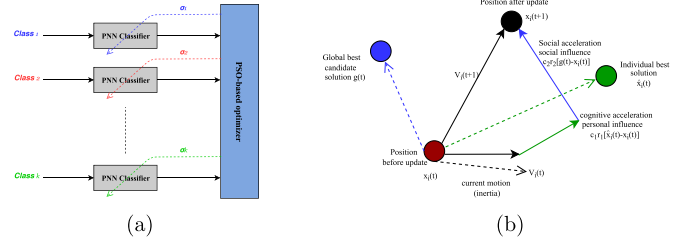


Fig. 6. Parameter (σ) optimization in PSO.

the velocity update can be performed through the following equation (Clerc and Kennedy, 2002):

$$v_i(t+1) = \chi [v_i(t) + c_1 r_1 (\hat{x}_i(t) - x_i(t)) + c_2 r_2 (\hat{x}_{g_i}(t) - x_i(t))] \quad (8)$$

Here, the parameter χ that determines the different version of PSO is called the constriction factor. Mathematically, equations (7) and (8) are equivalent, although there are considerable differences when selecting the corresponding parameters. The constriction factor is extracted using the following formula (Clerc and Kennedy, 2002):

$$\chi = \frac{2k}{|2 - \phi - \sqrt{\phi^2 - 4\phi}|} \quad (9)$$

Algorithm 1 Pseudocode for the PSO-PNN algorithm

- 1: Inputs for PNN: Normalized training dataset S_{train} and testing dataset S_{test}
- 2: Inputs for PSO: Swarm size (NP), c_1 , c_2 , χ using Eq (9), bounding box $\beta = [0, 1]$
- 3: Train the PNN by using all patterns in S_{train}
- 4: $i \leftarrow 0$, $i = 1, \dots, p$
- 5: Initialize a swarm, $\sigma_i(t) \in \beta$, $v_i(t) \in \beta$
- 6: Initialize best positions, $\hat{x}_i(t)$ and the index g
- 7: **repeat**
- 8: Update particle velocities, $v_i(t+1)$ using Eq (7) or (8)
- 9: Update particle position, $\sigma_i(t+1) = \sigma_i(t) + v_i(t+1)$
- 10: Constrain each $\sigma_i(t+1)$ in β particle
- 11: Compute $f(\sigma_i(t+1))$ proportion on S_{train}
- 12: Update best position $\hat{x}_i(t+1)$ using Eq (6) and the index g
- 13: Update iteration counter, $t = t + 1$
- 14: **until** (the error goal or the maximum number of iterations is not reached)
- 15: Write the best optimized spread parameter σ_g
- 16: Load all feature vectors in S_{test}
- 17: Concatenate all feature vectors using Eq (3) and the spread parameter σ_g calculated in step 15
- 18: Compute the summation of neuron using Eq (4) and satisfaction condition w_{ki}
- 19: Write the winning neuron using Eq (5)
- 20: **if** G_1 wins **then**
- 21: The behaviour of accessed user is *Normal*
- 22: **else if** any-other neurons from G_2 to G_{10} win **then** the behaviour of accessed user is *Malicious* (G_2 : Analysis, G_3 : Backdoors, G_4 : DoS, G_5 : Exploits, G_6 : Fuzzers, G_7 : Generic, G_8 : Reconnaissance, G_9 : Shellcode, G_{10} : Worms)
- 23: **end if**

4.2.3. Stability of constriction factor

Regarding the stability of the constriction factor χ , an ideal structure has been reported in (Clerc and Kennedy, 2002). In this case $\phi > 4$, where $\phi = c_1 + c_2$, and $k = 1$; the inertia weight ω is usually determined empirically. Furthermore, the initial parameters of swarm and

velocity are assigned randomly and uniformly. In this report, an initial value close to 1 that gradually decreases to zero is considered a good configuration (Parsopoulos and Vrahatis, 2002). In another similar experimental work with implementations of the particle swarm, the most standard version uses $\omega = 0.7298$ and $c_1 = c_2 = \chi/2$, where $\chi = 2.9922$ (Clerc and Kennedy, 2002).

Algorithm 1 describes the particle swarm optimization probabilistic neural network (PSO-PNN). In this algorithm, there are 2 different inputs for PSO and PNN, and these are input separately. The training and testing datasets used for PNN were explained in Section 5.

The key point for optimizing the PNN is to find the best spread parameters as the weight of hidden layer. As Fig. 6 (a) shown, each optimized PNN classifier will output one probabilistic value. Hence, the average value is obtained by k probabilistic values $(P_1, P_2, \dots, P_k)$. Finally, the average value determines the class of every input pattern. Fig. 5 shows the hybrid version of the PSO-PNN system with more details.

5. Experimental evaluation

In order to assess the efficiency and effectiveness of our technique in detecting and recognizing user malicious behaviours, we implemented the proposed approach on a state-of-the-art dataset.

5.1. Study setup and dataset

The dataset used to evaluate our proposed approach was collected via the IXIA PerfectStorm tool in the Cyber Range Lab of the Australian Centre for Cyber Security (ACCS) (MoustafaJill Slay, 2015a). The UNSW-NB15 consists of a mixture of real modern normal evidences and synthetic contemporary attack instances in the form of numerous records of users; it includes normal observations and nine families of attacks. Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode and Worms are the malicious behaviours; they are characterized by 47 features observed in real-world traffic network packets. These features are divided into five categories: flow, basic, content, time and generated features; these categories are depicted in Table 5. If we look more closely at the dataset, we can observe 2,540,044 records (100 GB) of raw network traffic (e.g., Pcap files) that have been logged using various devices such as mobiles, cameras and scanners. This dataset contains a total of 700,000 samples, including 677,789 normal and 22,211 malicious behaviours (MoustafaJill Slay, 2015a). In the UNSW-NB15, the IXIA traffic generator is connected to 3 different servers; servers 1 and 3 are used to generate normal behaviours, and server 2 is used to generate malicious behaviours. All servers are connected to two routers; pcap files are collected using router 1 directly and router 2 via firewall. A conceptual diagram of the UNSW-NB15 dataset is presented in Fig. 7.

Different training and testing sets constructed for each experiment; these are described in detail in their corresponding sections. In the first experiment, the system deals with a binary classification problem (detection), while in the second experiment, recognition of malicious behaviours is addressed as a multi-class classification problem.

5.2. Experimental results

5.2.1. Evaluation objectives

Regarding the evaluation criteria and the performance of the proposed approach in recognizing normal and malicious users' behaviours, we examined two complementary aspects; these are reflected in the following research questions:

- RQ1 (Detection): To what extent can the proposed approach detect malicious behaviours of users in the system?
- RQ2 (Recognition): To what extent can malicious behaviours that are automatically logged in the form of vectors of features be cor-

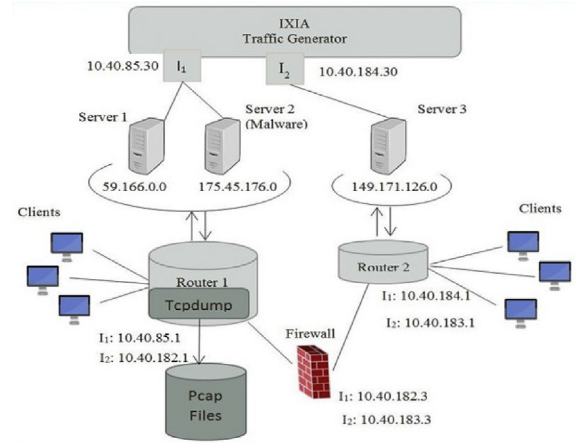


Fig. 7. A conceptual diagram for the UNSW-NB15 dataset (MoustafaJill Slay, 2015b).

rectly classified as belonging to various types of attacks?

RQ1 addresses the issue of detecting normal versus malicious behaviours. The first important point at the security level is: is the current logged entity a normal user or not? At this level, the administrator can detect and take action against users who generate an abnormal or partially malicious observation.

RQ2 refers to the recognition and classification of different behavioural abnormalities; here, the concern is the type of attack. Thus, in this stage of identification, once the malicious demeanour is recognized, the approach classifies the type of abnormality based on the types of attacks described in Section 3.

5.2.2. Evaluation metrics

Experiments are conducted on the UNSW-NB15 dataset to assess the performance and effectiveness of the PSO-PNN technique using well-known evaluation metrics such as True Positive Rate (TPR), False Positive Rate (FPR), Precision, Recall and F-measure; these metrics are calculated based on the four important retrieval parameters true negative (TN), false positive (FP), true positive (TP) and false negative (FN). TP is the number of actual malicious evidences classified as types of attacks, TN is the number of actual malicious evidences classified as normal, FN is the number of actual normal users classified as normal, and FP refers to the number of actual normal users who are classified as a type of attack. These metrics are explained below.

- **True Positive Rate (TPR)**, or Recall, is the fraction of malicious evidences correctly classified as attacks; it is computed using the following formula Eq. (10):

$$TPR = Recall = (TP) / ((TP) + (FN)) \quad (10)$$

- **False Positive Rate (FPR)**, is the fraction of normal behaviour patterns that are incorrectly classified as a type of attacks. The Eq. (11) describes how FPR is computed:

$$FPR = (FP) / ((TN) + (FP)) \quad (11)$$

- **Precision**, is the fraction of patterns that are recognized correctly; it is calculated in Eq. (12):

$$Precision = (TP) / ((TP) + (FP)) \quad (12)$$

- **F-measure**, the harmonious mean of precision and recall, is a statistical function for estimating the accuracy of a system by computing its precision and recall. It is given by Eq. (13):

Table 6
Confusion matrix for RQ1.

		Normal behaviour	Malicious behaviour
		C1	C2
Normal behaviour	C1	1000	0
Malicious behaviour	C2	40	960

Table 7
Performance measurements for RQ1.

	Normal behaviour C1	Malicious behaviour C2	Total
True Positive Rate	%100	%96	%98
False Positive Rate	0	%4	%2

$$F_{measure} = (2 * Precision * Recall) / (Precision + Recall) \quad (13)$$

The F-measure is a strong metric when the classification problem includes unbalanced classes or target values.

5.2.3. Evaluation results

Concerning the evaluation of RQ1 (detection), we took into account only two types of classes of normal and malicious behaviours. Hence, to train the model, we used a dataset with 4000 samples consisting of 2000 normal and 2000 malicious samples. To assess the obtained model using the proposed approach, we generated a testing dataset containing 2000 patterns by randomly selecting 1000 malicious and 1000 normal patterns. The confusion matrix depicted in Table 6 shows that the classification result is promising; we obtained overall values of 0.98 for TPR and 0.02 for FPR. The performance measurements for RQ1 are presented in Table 7. In order to evaluate RQ2 (recognition), we excluded the normal samples and focused on the malicious patterns that have been characterized for contemporary network attacks. Indeed, the type of attack is significantly important because each attack system defender may react differently. Thus, in this experiment, we aim to assess the ability of the proposed system to classify different types of attacks given the vector of features. We considered 4500 malicious patterns as the training set (a portion of the original dataset). Subsequently, 900 mali-

cious behaviours were considered as the test set.

Table 8 shows the confusion matrix that is obtained for all the attacks and Table 9 shows the corresponding performance measurements. It is clear from the confusion matrix that the proposed approach is capable of recognizing different types of attacks with high performance. Although the classification result for Back door attacks is lower than the results for other attacks (approximately 10%), it can still be considered an acceptable result based on the correct classification rate of 0.82. This result is due to the fact that the behavioural observations for class 1 and class 2 attacks are somewhat similar; thus, the false positive rates and false negative rates for these two types of attacks are slightly higher than the rates for other types of attacks. Our proposed method can thus be considered to have limited ability to properly distinguish between attacks 1 and 2.

5.3. Comparative study

In this section, we compare the performances of our proposed method with new state-of-the-art network-based intrusion detection techniques, such as Geometric Area Analysis Anomaly Detection System (GAA-ADS) (MoustafaJill Slay and Creech, 2017), Filter-based Support Vector Machine (FSVM) (Ambusaidi et al., 2016), Triangle Area Nearest Neighbours (TANN) (Tsai and Lin, 2010), Multivariate Correlation Analysis (MCA) (Tan et al., 2014), Euclidean Distance Map (EDM) (Tan et al., 2011), Computer Vision Techniques (CVT) (Tan et al., 2015) and Artificial Immune System (AIS) (Saurabh and Verma, 2016). Moreover, some new Cloud-based IDS including intrusion detection in Cloud computing (Patil et al., 2019) and Cloud-based malicious detection approach using statistical meta-features and system analysis engine (SAE) during the onset of Denial of Service (DoS) attacks (Watson et al., 2016) have been compared and discussed. A brief performance comparison of the existing detection techniques with the applied evaluation metrics are listed in Table 10.

Since the above-mentioned techniques depend on proximity measurements that are obtained by estimating the correlations between normal and abnormal instances and several malicious behaviours such as spy attacks (Giannetos and Dimitriou, 2013) significantly mimic normal behaviours, they interfere with the detection of normal samples and decrease the accuracy of the detection engine. The FSVM and methods were designed to learn from normal and malicious observations based on the concept of rule-based learning; in such methods, the

Table 8
Confusion matrix for RQ2.

	Analyses	Back door	DoS	Exploits	Fuzzers	Generic	Reconnaissance	Shellcode	Worms
	C1	C2	C3	C4	C5	C6	C7	C8	C9
Analyses	C1	99	0	0	1	0	0	0	0
Back door	C2	14	82	4	0	0	0	0	0
Dos	C3	0	0	96	0	0	0	2	0
Exploits	C4	0	0	0	100	0	0	0	0
Fuzzers	C5	0	0	0	0	98	1	0	1
Generic	C6	1	1	0	0	0	97	1	0
Reconn ...	C7	0	0	0	0	1	0	99	0
Shellcode	C8	0	0	0	1	1	0	0	98
Worms	C9	1	0	0	0	1	1	0	97

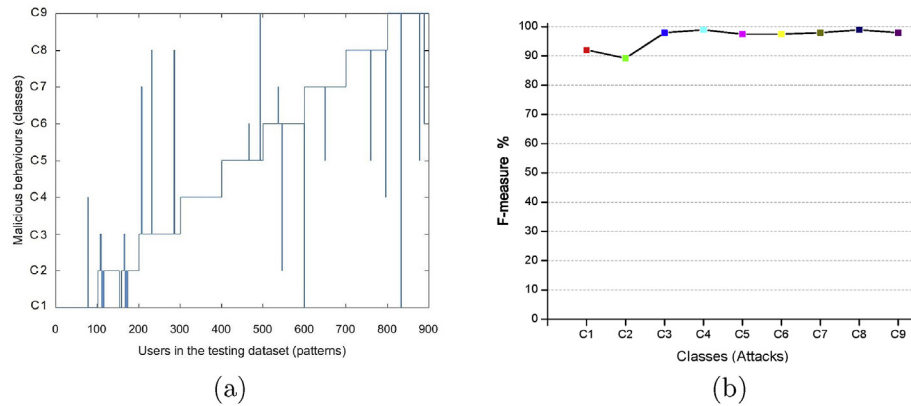
Table 9
Performance measurements for RQ2.

Metrics	Analyses C1	Back door C2	DoS C3	Exploits C4	Fuzzers C5	Generic C6	Reconn C7	Shellcode C8	Worms C9	Total
True Positive Rate	%99	%82	%96	%100	%98	%97	%99	%98	%97	%96.2
False Positive Rate	%16	%01	%04	%02	%03	%02	%03	%02	%01	%3.77
Precision	%86	%98	%100	%98	%97	%98	%97	%100	%99	%97
F-measure	%92	%89.2	%97.9	%98.9	%97.4	%97.4	%97.9	%98.9	%97.9	%96.38

Table 10

Comparative results with new network and Cloud-based anomaly detection techniques.

Approach	Objective	F	DR	FPR	P	F _m
Computer Vision Techniques (CVT) (Tan et al., 2015)	Denial-of-service attack detection	p	%95.3	%5.6	N/R	N/R
Triangle Area Nearest Neighbours (TANN) (Tsai and Lin, 2010)	Attack classification system	P	%91.1	%9.4	N/R	N/R
Artificial Immune System (AIS) (Saurabh and Verma, 2016)	Anomaly detection system	F	%90.4	%9.7	N/R	N/R
Filter-based Support Vector Machine(FSVM) (Ambusaidi et al., 2016)	Network intrusion detection system	P	%92.2	%8.7	N/R	N/R
Geometric Area Analysis (MoustafaJill Slay and Creech, 2017)	Anomaly detection system	F	%91.8	%5.8	N/R	N/R
Multivariate Correlation Analysis (Tan et al., 2014)	Denial-of-service attack detection	P	%95.1	%1.26	N/R	N/R
PSO-PNN	Malicious behaviour recognition	F	%96.4	%3.6	%96.4	%97.5

Note: F: Features (F:full, P:partial), DR: Detection Rate, FPR: False Positive Rate, P: Precision, F_m: F-measure, N/R: No available results.**Fig. 8.** (a) Classification distribution of the testing dataset, (b) F-measure rates for all classes (Attacks) in the testing dataset.

system always requires a huge number of observations to accurately understand the difference between normal and abnormal patterns, and this requirement is very arduous to achieve in real-time network traffic. Evaluation of these methods showed that their calculated detection rates for DoS and Probe attacks are higher than those achieved using our method but that is not true for other types of attacks.

The performance results shown in Table 10 clearly demonstrate the superiority of the PSO-PNN technique in terms of Recall, Precision and F-measure when used on the UNSW-NB15 dataset. Although the MCA, EDM, CVT, and TANN techniques generally performed well and reported high accuracy, these techniques were designed to detect only DoS attacks, not the other malicious behaviours described in Section 3. Ultimately, the PSO-PNN technique can achieve better performance than other techniques for malicious behaviours in real-time network traffic and can efficiently discriminate between normal and malicious behaviours.

6. Discussion and conclusion

It is almost confirmed that users' credentials are not sufficient to prevent attacks. Today, it is necessary for an authentication system to monitor and assess users' activities within the system.

In this study, we proposed a system that uses significant features of activity to model normal and malicious behaviour of users in Cloud-based environments and represents a mathematical and understand-

able format for a learning machine. A self-optimized machine learning approach based on a Probabilistic Neural Network (PNN) was used to train the network and to allow it to learn from patterns of activity associated with different types of attacks. In addition, a Particle Swarm Optimization (PSO) algorithm was combined with the Probabilistic Neural Network (PSO-PNN) to optimize the performance of the system and reduce the classification errors. Using this collaborative system, we are able to recognize and classify users' behaviours as either normal activity or various types of modern attacks. Our experimental results demonstrate that the proposed method is promising.

Fig. 8 (a) shows the classification distribution of 900 patterns in the testing dataset, the Y-axis shows the different classes from C1 to C9 (attacks) in testing dataset and X-axis shows how patterns in the testing dataset are classified to their belonging classes (attacks). The horizontal blue lines show correct classifications, for instance, in Fig. 8 (a) the blue straight line from 0 to 100 shows the first 100 patterns in the testing dataset are belong to class 1 (Analyses attack). The turbulences (vertical blue lines) accrue in each class when there is misclassified pattern. Fig. 8 (b) represents the F-measure rate for every class separately. The performance measurements provided in Table 9 prove that the results achieved using the PSO-PNN approach provide high accuracy for recognizing malicious behaviours in emerging computing environments. The aim of the evaluation was to assess the results taken by the proposed approach based on the following key aspects:

- RQ1: detection of normal and malicious behaviours.

Table 11

Comparisons of feature vectors based on intra-class similarity and inter-class dissimilarity in Backdoors and DoS attacks.

Patterns	Extracted features
Backdoors pattern	(0,0,0.000005,200,0,254,0,0,2,0,0,0,100,0,0,0,0,0.011,0,0,0,0,2,0,0,0,23,18,6,8,6,6,6)
Backdoors pattern	(0,0,0.000004,200,0,254,0,0,2,0,0,0,100,0,0,0,0,0.005,0,0,0,0,2,0,0,0,22,17,4,6,4,4,4)
DoS pattern	(39500,80,0.17,1214,268,254,252,2,1,10,6,255,255,121,45,1,0,1020,64,62,19,0.05,0.01,0.045,0,1,1,0,0,5,2,1,1,1,1)
DoS pattern	(23910,80,2.6,2082,268,254,252,6,1,14,6,255,255,149,45,1,0,17309,1007,198,509,0.09,0.05,0.05,0,1,1,0,0,5,2,1,2,2,1,1)

- RQ2: recognition of different types of contemporary network attacks (malicious behaviours) as a multi-class classification problem.

The result of RQ1 indicates that 100 percent of normal users are correctly classified as normal and 96 percent of malicious users are correctly classified as malicious. The remaining 4 percent are true negatives in which malicious users are incorrectly classified as normal users.

In RQ2, we have shown that the proposed system is not only able to detect normal and abnormal behaviour but that it is even capable of classifying different types of malicious behaviours with high performance. The PSO-PNN system can statistically find high intra-class similarity for the observations inside every class and recognize the new patterns with similar observations. In an opposite way, these statistical properties give capability to the system to find inter-class dissimilarity among different classes (attacks). However, in some cases, the system encountered highly similar abnormal activities (e.g., attacks 1 and 2) that affected the performance of recognition.

Table 11 shows an example of the similarity among the patterns inside class 2 (high intra-class similarity in Backdoors attack observations) as well as class 3 (DoS attack), and the dissimilarity between class 2 and 3 (inter-class dissimilarity). In addition, a tree diagram regarding the differences among all attacks based on attack categories, attack sub-categories and the number of events for every sub-category is illustrated in Fig. 2. In modelling the behaviour of users within a system, extracted features and observations play a crucial role since those features attest to the actual intention of the users. Thus, it is necessary to carefully and accurately extract the most important features that characterize the users' demeanour. The use of advanced tools to capture the raw data from network traffic packets and deep learning approaches to deal with high dimensionality problems, as well as derivation of the significant features, would help increase the intra-class similarity of the samples.

As part of our future work, we plan to extend the proposed approach using deep learning techniques and new abnormal observations in Zero-day attacks and vulnerabilities. We believe that the use of fewer samples with rich numbers of features often leads to an ideal recognition system.

Acknowledgements

The authors would like to thank the anonymous reviewers for their helpful comments. This work was supported by the National Natural Science Foundation of China (611770035, 61272420, 81674099, 61502233), the Fundamental Research Fund for the Central Universities (30916011328, 30918015103), the Nanjing Science and Technology Development Plan Project (201805036), and the "13th Five-Year" Equipment Field Fund (61403120501).

References

- Abdi, Herv, Williams, Lynne J., 2010. Principal component analysis. Wiley interdisciplinary reviews: Comput. Stat. 2 (4), 433–459.
- Alouane, Meryeme, El Bakkali, Hanan, 2015. Security, privacy and trust in cloud computing: a comparative study. In: Cloud Technologies and Applications (CloudTech), 2015 International Conference on. IEEE, pp. 1–8.
- Ambusaidi, Mohammed A., He, Xiangjian, Nanda, Priyadarsi, Tan, Zhiyuan, 2016. Building an intrusion detection system using a filter-based feature selection algorithm. IEEE Trans. Comput. 65 (10), 2986–2998.
- Binu, Sumitra, Misbahuddin, Mohammed, 2013. A survey of traditional and cloud specific security issues. In: International Symposium on Security in Computing and Communication. Springer, pp. 110–129.
- Bonomi, Flavio, Milito, Rodolfo, Jiang, Zhu, Addepalli, Sateesh, 2012. Fog computing and its role in the internet of things. In: Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing. ACM, pp. 13–16.
- Cen, Lei, Gates, Christopher S., Luo, Si, Li, Ninghui, 2015. A probabilistic discriminative model for android malware detection with decompiled source code. IEEE Trans. Dependable Secure Comput. 12 (4), 400–412.
- Clerc, Maurice, Kennedy, James, 2002. The particle swarm-explosion, stability, and convergence in a multidimensional complex space. IEEE Trans. Evol. Comput. 6 (1), 58–73.
- Eberhart, Russell C., Shi, Yuhui, Kennedy, James, 2001. Swarm Intelligence. Elsevier.
- Giannetsos, Thanassis, Dimitriou, Tassos, 2013. Spy-sense: spyware tool for executing stealthy exploits against sensor networks. In: Proceedings of the 2nd ACM Workshop on Hot Topics on Wireless Network Security and Privacy. ACM, pp. 7–12.
- Han, Weijie, Xue, Jingfeng, Wang, Yong, Liu, Zhenyan, Kong, Zixiao, 2019. Malinsight: a systematic profiling based malware detection framework. J. Netw. Comput. Appl. 125, 236–250.
- Hoefler, Christina N., Karagiannis, Georgios, 2010. Taxonomy of cloud computing services. In: GLOBECOM Workshops (GC Wkshps), 2010 IEEE. IEEE, pp. 1345–1350.
- Inoue, Daisuke, Yoshioka, Katsunari, Eto, Masashi, Hoshizawa, Yuji, Nakao, Koji, 2009. Automated malware analysis system and its sandbox for revealing malware's internal and external activities. IEICE Trans. Info Syst. 92 (5), 945–954.
- Jouini, Mouna, Arfa Rabai, Latifa Ben, 2019. A security framework for secure cloud computing environments. In: Cloud Security: Concepts, Methodologies, Tools, and Applications. IGI Global, pp. 249–263.
- Kang, Inho, Jeong, Myong K., Kong, Dongjoon, 2012. A differentiated one-class classification method with applications to intrusion detection. Expert Syst. Appl. 39 (4), 3899–3905.
- Keshk, Marwa, Moustafa, Nour, Elena, Sitnikova, Creech, Gideon, 2017. Privacy preservation intrusion detection technique for scada systems. In: Military Communications and Information Systems Conference (MilCIS), 2017. IEEE, pp. 1–6.
- Kholidi, Hisham A., Baiardi, Fabrizio, 2012. Cids: a framework for intrusion detection in cloud systems. In: 2012 Ninth International Conference on Information Technology-New Generations. IEEE, pp. 379–385.
- Kirda, Engin, Kruegel, Christopher, Banks, Greg, Vigna, Giovanni, Kemmerer, Richard, 2006. Behavior-based spyware detection. In: Usenix Security Symposium, p. 694.
- Kumar, Manish, Hanumanthappa, M., Suresh Kumar, T.V., 2013. Encrypted traffic and ipsec challenges for intrusion detection system. In: Proceedings of International Conference on Advances in Computing. Springer, pp. 721–727.
- Martignoni, Lorenzo, Paleari, Roberto, Bruschi, Danilo, 2009. A framework for behavior-based malware analysis in the cloud. In: International Conference on Information Systems Security. Springer, pp. 178–192.
- Mell, Peter, Tim Grance, et al., 2011. The Nist Definition of Cloud Computing.
- Mithal, Tulika, Shah, Kshitij, Singh, Dushyant Kumar, 2016. Case studies on intelligent approaches for static malware analysis. In: Emerging Research in Computing, Information, Communication and Applications. Springer, pp. 555–567.
- Mohamed, Belaoued, Mazouzi, Smaïne, 2015. A real-time pe-malware detection system based on chi-square test and pe-file features. In: IFIP International Conference on Computer Science and its Applications x000D. Springer, pp. 416–425.
- Monowar, H Bhuyar, Bhattacharyya, Dhruva Kumar, Kalita, Jugal K., 2014. Network anomaly detection: methods, systems and tools. Ieee communications surveys & tutorials 16 (1), 303–336.
- Moonsamy, Veelasha, Tian, Ronghua, Lynn, Batten, 2011. Feature reduction to speed up malware classification. In: Nordic Conference on Secure IT Systems. Springer, pp. 176–188.
- Moon, Daesung, Sung, Bum Pan, Kim, Ikkyun, 2016. Host-based intrusion detection system for secure human-centric computing. J. Supercomput. 72 (7), 2520–2536.
- Moustafa, Nour, Hu, Jiankun, Jill Slay, 2019. A holistic review of network anomaly detection systems: a comprehensive survey. J. Netw. Comput. Appl. 128, 33–55.
- Moustafa, Nour, Jill Slay, 2015. The significant features of the unsw-nb15 and the kdd99 data sets for network intrusion detection systems. In: Building Analysis Datasets and Gathering Experience Returns for Security (BADGERS), 2015 4th International Workshop on. IEEE, pp. 25–31.
- Moustafa, Nour, Jill Slay, 2015. Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set). In: 2015 Military Communications and Information Systems Conference (MilCIS). IEEE, pp. 1–6.
- Moustafa, Nour, Jill Slay, Creech, Gideon, 2017. Novel Geometric Area Analysis Technique for Anomaly Detection Using Trapezoidal Area Estimation on Large-Scale Networks. IEEE Transactions on Big Data.
- Parsopoulos, K.E., Vrahatis, M.N., 2002. Initializing the particle swarm optimizer using the nonlinear simplex method. Advances in intelligent systems, fuzzy systems, evolutionary computation 216, 1–6.
- Patil, Rajendra, Dudeja, Harsha, Modi, Chirag, 2019. Designing an efficient security framework for detecting intrusions in virtual network of cloud computing. Comput. Secur. 85, 402–422.
- Peng, Jian, Raymond Choo, Kim-Kwang, Ashman, Helen, 2016. User profiling in intrusion detection: a review. J. Netw. Comput. Appl. 72, 14–27.
- Reza Zargar, Gholam, Kabiri, Peyman, 2009. Identification of effective network features for probing attack detection. In: 2009 First International Conference on Networked Digital Technologies. IEEE, pp. 392–397.
- Rieck, Konrad, Holz, Thorsten, Willems, Carsten, Dssell, Patrick, Laskov, Pavel, 2008. Learning and classification of malware behavior. In: International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment. Springer, pp. 108–125.
- Russell, Eberhart, Kennedy, James, 1995. A new optimizer using particle swarm theory. In: Micro Machine and Human Science, 1995. MHS'95., Proceedings of the Sixth International Symposium on. IEEE, pp. 39–43.
- Saber, Mohammed, El Farissi, Ilhame, Chadli, Sara, Mohamed, Emharraf, Ghaouth Belkasm, Mohammed, 2017. Performance analysis of an intrusion detection systems based of artificial neural network. In: Europe and MENA Cooperation Advances in Information and Communication Technologies. Springer, pp. 511–521.
- Sait, Saad Y., Bhandari, Akshay, Khare, Shreya, Cyriac, Murthy, Hema A., 2015. Multi-level anomaly detection: relevance of big data analytics in networks. Sadhana 40 (6), 1737–1767.
- Saurabh, Praneet, Verma, Bhupendra, 2016. An efficient proactive artificial immune system based anomaly detection and prevention system. Expert Syst. Appl. 60, 311–320.

- Shi, Weisong, Cao, Jie, Zhang, Quan, Li, Youhuizi, Xu, Lanyu, 2016. Edge computing: vision and challenges. *IEEE Internet of Things Journal* 3 (5), 637–646.
- Specht, Donald F., 1990. Probabilistic neural networks. *Neural Netw.* 3 (1), 109–118.
- Sun, Ming-Kung, Lin, Mao-Jie, Chang, Michael, Lai, Chi-Sung, Lin, Hui-Tang, 2011. Malware virtualization-resistant behavior detection. In: *Parallel and Distributed Systems (ICPADS)*, 2011 IEEE 17th International Conference on. IEEE, pp. 912–917.
- Tan, Zhiyuan, Jamdagni, Aruna, He, Xiangjian, Nanda, Priyadarsi, Ren, Ping Liu, 2011. Denial-of-service attack detection based on multivariate correlation analysis. In: *International Conference on Neural Information Processing*. Springer, pp. 756–765.
- Tan, Zhiyuan, Jamdagni, Aruna, He, Xiangjian, Nanda, Priyadarsi, Ren, Ping Liu, 2014. A system for denial-of-service attack detection based on multivariate correlation analysis. *IEEE Trans. Parallel Distrib. Syst.* 25 (2), 447–456.
- Tan, Zhiyuan, Jamdagni, Aruna, He, Xiangjian, Nanda, Priyadarsi, Ren, Ping Liu, Hu, Jiankun, 2015. Detection of denial-of-service attacks based on computer vision techniques. *IEEE Trans. Comput.* 64 (9), 2519–2533.
- Tian, Li-qin, Lin, Chuang, Ni, Yang, 2010. Evaluation of user behavior trust in cloud computing. In: *Computer Application and System Modeling (ICCASM)*, 2010 International Conference on, vol. 7. IEEE, pp. V7–V567.
- Tsai, Chih-Fong, Lin, Chia-Ying, 2010. A triangle area based nearest neighbors approach to intrusion detection. *Pattern Recognit.* 43 (1), 222–229.
- Watson, Michael R., Marnerides, Angelos K., Mauthe, Andreas, Hutchison, David, et al., 2016. Malware detection in cloud computing infrastructures. *IEEE Trans. Dependable Secure Comput.* 13 (2), 192–205.
- Zhang, Mu, Duan, Yue, Yin, Heng, Zhao, Zhiruo, 2014. Semantics-aware android malware classification using weighted contextual api dependency graphs. In: *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. ACM, pp. 1105–1116.
- Zhang, Yuan, Xu, Chunxiang, Li, Hongwei, Yang, Kan, Zhou, Jianying, Lin, Xiaodong, 2018. Healthdep: an efficient and secure deduplication scheme for cloud-assisted ehealth systems. *IEEE Transactions on Industrial Informatics* 14 (9), 4101–4112.
- Zhang, Yuan, Xu, Chunxiang, Lin, Xiaodong, Shen, Xuemin Sherman, 2019. Blockchain-based public integrity verification for cloud storage against procrastinating auditors. *IEEE Transactions on Cloud Computing*.
- Zhao, Zongqu, Wang, Junfeng, Bai, Jinrong, 2014. Malware detection method based on the control-flow construct feature of software. *IET Inf. Secur.* 8 (1), 18–24.



Mahdi Rabbani is currently a Ph.D student in the School of Computer Science and Engineering in Nanjing University Of Science and Technology (NJUST), Nanjing, China. He completed his Master of technology (M.tech) in University of Mysore, India and holding bachelor degree in software engineering in Iran. Prior to this, he worked as Research Scholar in University of Mysore in India for 2 years. His research interests include network security in new computing technologies, artificial intelligence, collaborative intrusion detection, trust computing.



Yongli Wang received the Ph.D. degree from the South-east University in 2006. He is currently a professor in the Department of Computer Science and Engineering at Nanjing University of Science and Technology, China. He is the author or coauthor of more than 70 papers in database and leading pattern recognition journals and conferences. His research includes sensor networks, data streams management, mobile computing, data mining, cyber-physical system, and health care monitoring, and is supported by the NCSF and other agencies. For his research activities, he also spent extended periods



of time at Drexel University, USA, Brunel University, UK, and Fraunhofer Institutes, Germany, He is a member of the ACM and the IEEE.

Reza Khoshkangini received his Ph.D degree in Computer Science from University of Padova, Italy. He completed his Master of technology (M.tech) in University of Mysore, India and holding bachelor degree in software engineering in Iran. His research interests include network security in new computing technologies, artificial intelligence, collaborative intrusion detection, trust computing.



Hamed Jelodar is received the Master Degree in computer software engineering from Islamic Azad University, Bushehr, Iran, in 2014. He currently is a PHD Candidate in the School of Computer Science, Nanjing University of Science and technology, Nanjing, China. His research focuses on data mining, wireless networks, fuzzy system, machine Learning.



Ruxin Zhao received a bachelor of engineering degree at the College of computer science and engineering at Northeastern Petroleum University, China, in 2014. In 2017, he received a master's degree in engineering at the school of information science and engineering at the Guangxi University for Nationalities. At present, he is a Ph.D. candidate at the school of computer science and engineering at Nanjing University of Science and Technology. His main research interest is swarm intelligence optimization algorithm.



Peng Hu, he is a Ph.D. student in the department of Computer science & engineering of Nanjing University of Science and Technology. His main research interests are machine learning and information Security.